

Great! Let's simulate a real interview session. Based on your resume, I'll ask you 100 questions—divided into the following categories:

Breakdown:

1. **Basic HR Questions (10)**
 2. **General Technical (10)**
 3. **Python (10)**
 4. **Web Development – HTML/CSS/JS (10)**
 5. **React.js (5)**
 6. **Backend (Django/Flask) (10)**
 7. **Databases & SQL (10)**
 8. **Projects (Urban ZoomIn, QueryManager) (10)**
 9. **Tools & Platforms (JIRA, GitHub, PWA, Streamlit) (5)**
 10. **Business & Startup Mindset (10)**
 11. **Advanced & Situational Thinking (10)**
-

1. Basic HR Questions (with sample answers)

1. **Tell me about yourself.**
I'm Srinadh Chintakindi, a B.Tech CSE student passionate about building real-world software solutions. I founded Urban ZoomIn—a hyperlocal platform to connect users to nearby stores using Flask and MySQL. I also love open-source and built QueryManager for database automation.
2. **Why should we hire you?**
I bring a blend of technical and business thinking. My projects show initiative, and I'm comfortable taking products from idea to deployment. I'm not just a coder—I understand user needs and iterate quickly.
3. **What are your strengths?**
Problem-solving, adaptability, public speaking, and full-stack web development.
4. **What's your weakness?**
I tend to overcommit to projects I'm passionate about, but I'm learning to balance priorities better.

5. **Where do you see yourself in 5 years?**
As a tech leader solving real-world problems, possibly leading a product team or a startup.
 6. **Tell me about a challenge you faced.**
Convincing local shopkeepers to adopt Urban ZoomIn was hard. I explained the value, demoed the product, and improved based on feedback.
 7. **What are your hobbies?**
Exploring new tech, writing blogs, and mentoring peers in coding.
 8. **Describe your leadership style.**
I lead by example—take ownership, communicate clearly, and always keep user experience in mind.
 9. **What motivates you?**
Creating impact—when my code solves a real user problem, it's deeply rewarding.
 10. **Are you open to relocation?**
Yes, I'm flexible and open to both on-site and hybrid roles.
-

2. General Technical Questions

11. **What is an API?**
An API is an interface that allows two applications to communicate. For example, REST APIs help frontend apps fetch or send data to the backend.
12. **Explain the difference between Git and GitHub.**
Git is a version control system; GitHub is a cloud-based hosting service for Git repositories.
13. **What is the difference between frontend and backend?**
Frontend handles UI/UX; backend handles server logic, database, and APIs.
14. **What is HTTP vs HTTPS?**
HTTP is unsecured; HTTPS adds a security layer using SSL/TLS encryption.
15. **What is a PWA?**
A Progressive Web App behaves like a native app but is built using web technologies.
16. **Difference between PUT and POST?**
POST is for creating, PUT is for updating an existing resource.
17. **What is MVC architecture?**
Model-View-Controller. Separates data, UI, and business logic for better design.
18. **What are cookies and sessions?**
Cookies store data on the client-side; sessions on the server.

19. **What is deployment?**

Making your application live and accessible to users.

20. **Explain CRUD.**

Create, Read, Update, Delete—basic database operations.

3. Python Questions

21. **What are Python's data types?**

int, float, string, list, tuple, set, dict, etc.

22. **Difference between list and tuple?**

Lists are mutable; tuples are immutable.

23. **What is a decorator?**

A function that modifies the behavior of another function.

24. ****What are args and kwargs?**

*_args passes variable number of non-keyword arguments; **kwargs for keyworded args._*

25. **What is a lambda function?**

An anonymous one-liner function: `lambda x: x + 1`

26. **Explain list comprehension.**

A concise way to create lists: `[x for x in range(5)]`

27. **What are Python modules and packages?**

Modules are files with Python code; packages are directories with `__init__.py`.

28. **What is exception handling?**

try-except blocks to handle runtime errors.

29. **Difference between is and ==?**

is checks identity; == checks equality.

30. **What is PEP8?**

Python's style guide for writing clean code.

4. Web Development – HTML/CSS/JS

31. **What is semantic HTML?**

Using HTML tags that convey meaning, like `<article>`, `<section>`.

32. **Difference between id and class?**
id is unique; class can be reused.
33. **What are Flexbox and Grid?**
CSS layouts to arrange items. Flexbox for 1D, Grid for 2D layouts.
34. **What is the DOM?**
Document Object Model—a tree-like structure of HTML elements.
35. **What are CSS selectors?**
They target elements for styling, e.g., `.class`, `#id`, `div > p`.
36. **What is JavaScript hoisting?**
Variable and function declarations are moved to the top before execution.
37. **Difference between var, let, and const?**
var is function-scoped, let/const are block-scoped. const cannot be reassigned.
38. **What are JS Promises?**
They handle async operations and have `.then()` and `.catch()`.
39. **Explain event bubbling.**
Events propagate from the innermost to outermost element.
40. **What are ES6 features?**
Arrow functions, destructuring, spread/rest, classes, etc.
-

5. React.js

41. **What is React?**
A JavaScript library for building UI using components.
42. **What is JSX?**
JS syntax extension allowing HTML in JS code.
43. **What is state vs props?**
State is internal to a component; props are passed from parent.
44. **What are hooks?**
Functions like `useState`, `useEffect` that manage state/lifecycle.
45. **What is a component?**
A reusable piece of UI.
-

✓ 6. Backend (Flask/Django)

- 46. **What is Flask?**
A lightweight Python web framework.
 - 47. **What is Django?**
A high-level framework with built-in admin, ORM, and security.
 - 48. **Flask vs Django?**
Flask = lightweight and flexible. Django = batteries-included.
 - 49. **What are routes in Flask?**
URLs that trigger specific Python functions.
 - 50. **What is Jinja?**
Flask's templating engine.
 - 51. **How do you connect Flask to MySQL?**
Using connectors like `mysql-connector-python` or `SQLAlchemy`.
 - 52. **How do you handle forms in Flask?**
Using `request.form`, `WTForms`, or simple HTML forms.
 - 53. **How do you deploy a Flask app?**
Using platforms like `PythonAnywhere`, `Heroku`, or `VPS`.
 - 54. **What is CSRF protection?**
Cross-Site Request Forgery—prevented using tokens.
 - 55. **What are middlewares in Django?**
Hooks that process requests/responses globally.
-

✓ 7. Database & SQL

- 56. **What is a relational database?**
A structured database with tables, rows, and relations.
- 57. **What is a foreign key?**
A key used to link two tables.
- 58. **Write a SQL query to get all records from a table.**
`SELECT * FROM tablename;`
- 59. **How to find duplicates in a table?**
Using `GROUP BY` and `HAVING COUNT(*) > 1`

60. **What is normalization?**
Removing data redundancy by organizing tables.
61. **Explain JOIN types.**
INNER, LEFT, RIGHT, FULL joins.
62. **What is indexing?**
Improves search speed in large datasets.
63. **How to update a record in SQL?**
`UPDATE table SET column=value WHERE condition;`
64. **What is a stored procedure?**
A saved SQL query you can run repeatedly.
65. **Explain ACID properties.**
Atomicity, Consistency, Isolation, Durability.
-

8. Projects (Urban ZoomIn, QueryManager)

66. **What is Urban ZoomIn?**
A local store discovery web app with real-time updates.
67. **What tech stack was used?**
Flask, MySQL, HTML/CSS/JS, deployed on PythonAnywhere.
68. **How did you onboard shopkeepers?**
Explained benefits, demoed app, took feedback.
69. **How did you implement real-time status?**
Used database updates and polling from frontend.
70. **What challenges did you face in Urban ZoomIn?**
Non-technical users, UX improvement, database optimization.
71. **What is QueryManager?**
A PyPI package to simplify and automate DB operations.
72. **Why did you build QueryManager?**
To avoid repetitive DB code during development.
73. **How do you publish to PyPI?**
Use `setup.py`, build with `build`, and upload using `twine`.
74. **How does it improve productivity?**
Reduces boilerplate DB code, improves reusability.

75. **Did you get user feedback?**
Yes—on GitHub and peers found it useful in mini-projects.
-

✓ 9. Tools & Platforms

76. **How do you use GitHub in projects?**
For version control, collaboration, and showcasing work.
77. **What is JIRA used for?**
Project and issue tracking in teams.
78. **What is Streamlit?**
Python framework for building quick web apps for ML/data.
79. **How did you build the PWA version?**
Used service workers, manifest.json, and HTTPS.
80. **What is PythonAnywhere?**
Cloud platform for hosting Python web apps.
-

✓ 10. Business & Startup Mindset

81. **How do you identify user pain points?**
Through observation, feedback, and interviews.
82. **What is grassroots marketing?**
Hyperlocal, low-cost marketing by direct community outreach.
83. **Explain the value proposition of Urban ZoomIn.**
Helps users quickly find nearby stores and their current status.
84. **How do you measure user engagement?**
Through metrics like bounce rate, time on page, and feedback.
85. **What makes a product successful?**
Solving real pain points + great UX + strong feedback loop.
86. **How do you pitch an idea?**
Start with the problem, explain the solution, and end with benefits.
87. **What makes a good MVP?**
Solves the core problem with minimal features.

88. What's your GTM (Go-To-Market) strategy?

Start locally, onboard early adopters, collect feedback, scale gradually.

89. What is product-market fit?

When users actively use and recommend your product.

90. How do you prioritize features?

Based on user demand, feasibility, and impact.

 11. Advanced & Situational Thinking

91. You find a bug in production. What do you do?

Check logs, recreate locally, fix, test, then deploy a patch.

92. You disagree with a teammate. What do you do?

Understand their view, suggest alternatives, agree on best for the user.

93. You have only 2 days to complete a feature. How do you handle it?

Break it into core + optional parts. Deliver core first.

94. What is technical debt?

Shortcuts in code that slow down future changes.

95. How do you learn new technologies?

Project-based learning, official docs, hands-on practice.

96. How do you ensure code quality?

Code reviews, linters, testing, following standards.

97. How do you handle feedback?

With an open mind—feedback helps me grow.

98. You need to explain your project to a non-tech person. How?

Use simple terms, analogies, and focus on outcomes.

99. What are you currently learning?

Deepening React and exploring AI APIs for integration.

100. One thing you'd improve in Urban ZoomIn?

Add real-time notifications and better search filtering.

Let me know if you'd like these in a **mock interview format**, **Notion tracker**, or **flashcard-style** for revision!